Code like this can feel a bit backward. The first thing it does is to check to see if what it's about to do has been done before. Like being a character in a sci-fi time travel story, you have to think fourth-dimensionally: what might have happened the first time so that the *second* time this code runs, it does the right thing?

Fortunately, more and more third parties provide an easier way to address this issue by allowing you to specify that an operation should be treated as idempotent on the third party's side using a special key.

Use an Idempotency Key if the Service Provides One

The problem you just solved with the email service also exists with the fulfillment service. Although the fake fulfillment service won't fulfill orders when you configure it to crash, you can imagine that in certain situations in the real world (such as a network outage), a retried job could send an order out more than once.

Unlike the email service, the fulfillment service allows you to specify an *idempotency key*. The service will guarantee that it will perform only one request with that key. If it gets a second request with the same key, it won't duplicate the work, but will return as if it had. The fulfillment service handles all the fourth-dimensional thinking and management for you, as outlined in the following diagram: